iOS program to ask user for two numbers and print the sum

```
import SwiftUI
struct ContentView: View {
    @State private var number1 = ""
    @State private var number2 = ""
    @State private var result = 0
    var body: some View {
        VStack(spacing: 20) {
            Text("Sum Calculator")
                .font(.largeTitle)
                .fontWeight(.bold)
            TextField("Enter first number", text: $number1)
                .keyboardType(.numberPad)
                .textFieldStyle(RoundedBorderTextFieldStyle())
            TextField("Enter second number", text: $number2)
                .keyboardType(.numberPad)
                .textFieldStyle(RoundedBorderTextFieldStyle())
            Button("Calculate Sum") {
                let num1 = Int(number1) ?? 0
                let num2 = Int(number2) ?? 0
                result = num1 + num2
            }
            .padding()
            .background(Color.blue)
            .foregroundColor(.white)
            .cornerRadius(8)
            Text("Result: \(result)")
                .font(.title2)
        }
        .padding()
    }
}
#Preview {
```

```
    ContentView()
}
```

iOS program for add one/subtract one application

```
import SwiftUI
struct ContentView: View {
    @State private var number = 0   // Keeps track of the number
    var body: some View {
        VStack(spacing: 30) {
            // Display number
            Text("\(number)")
                .font(.system(size: 60))
                .fontWeight(.bold)
            // Buttons
            HStack(spacing: 20) {
                Button("Add One") {
                    number += 1
                }
                .padding()
                .background(Color.green)
                .foregroundColor(.white)
                .cornerRadius(10)
                Button("Subtract One") {
                    number -= 1
                }
                .padding()
                .background(Color.red)
                .foregroundColor(.white)
                .cornerRadius(10)
            }
        }
        .padding()
    }
```

```
}
#Preview {
    ContentView()
}
```

iOS Program to display data in list

```
import SwiftUI
struct ContentView: View {
    // Array of data
    let items = ["Apple", "Banana", "Orange", "Mango", "Grapes"]
    var body: some View {
        NavigationStack {
            List(items, id: \.self) { item in
                Text(item)
            }
            .navigationTitle("Fruits")
        }
    }
}
#Preview {
    ContentView()
}
```

Old Questions

1. What is a Storyboard?

Definition:

A **storyboard** is a visual representation of the user interface of an iOS application. It allows developers to **design multiple screens (view controllers) and the navigation flow** between them in a single canvas.

Key Points:

- Provides a **drag-and-drop interface** to add UI components like buttons, labels, text fields, etc.
- Connects UI elements to **ViewController code** using IBOutlets and IBActions.
- Supports **segue connections** to navigate between screens.
- Helps visualize the **overall flow** of the application.

---

2. Swift Program to Find Largest and Smallest Number Using Ternary Operator

```swift
// Input three numbers
let num1 = 25
let num2 = 10
let num3 = 40
// Find largest using ternary operator
let largest = (num1 > num2) ? ((num1 > num3) ? num1 : num3) : ((num2 > num3) ? num2 : num3)
// Find smallest using ternary operator
let smallest = (num1 < num2) ? ((num1 < num3) ? num1 : num3) : ((num2 < num3) ? num2 : num3)
print("Largest number: ", largest)
print("Smallest number: ", smallest)
```

Explanation

- **Ternary Operator:** `condition ? valueIfTrue : valueIfFalse` allows compact conditional evaluation.
- **Largest Number Logic:** Compares `num1`, `num2`, and `num3` to find the maximum.
- **Smallest Number Logic:** Compares `num1`, `num2`, and `num3` to find the minimum.
- **Print Statement:** Displays the largest and smallest numbers in the console.

Conclusion

- **Storyboard:** Simplifies UI design and navigation flow in iOS apps.
- **Ternary Operator:** Provides a concise way to find the largest and smallest numbers without using multiple if-else statements.

  This Swift program demonstrates **efficient comparison logic** using ternary operators for three numbers.

3. View Hierarchy in iOS Programming

Definition:

View hierarchy is a **tree-like structure of UI elements** (views) in an iOS app. It determines how views are rendered, layered, and how they handle user interactions.

Key Points:

- **Root View:** The main container of all subviews (e.g., `UIViewController.view`).
- **Subviews:** Views added to the root view or other views, forming a hierarchy.
- **Parent-Child Relationship:** Views can have child views (subviews) and parent views.
- **Z-Order:** The order of views determines which view appears on top.
- **Event Handling:** Touch and gesture events are propagated through the view hierarchy.

Example of View Hierarchy:

- `UIViewController.view` (root)
  - `UILabel`
  - `UIButton`
  - `UIImageView`
    - `UIView` (subview inside image view)

4. Swift Function to Calculate Factorial

```
import Foundation
func factorial(_ n: Int) -> Int {
   if n == 0 || n == 1 {
```

```
        return 1
    } else {
        var result = 1
        for i in 2...n {
            result *= i
        }
        return result
    }
}
// Example usage
let number = 5
print("Factorial of $number) is $factorial(number))")
```

Explanation

- **Function Name:** `factorial` takes a positive integer `n`.
- **Base Case:** Returns 1 if `n` is 0 or 1.
- **Loop:** Multiplies numbers from 2 to `n` to compute factorial.
- **Return:** Returns the final factorial value.

Conclusion

- **View Hierarchy:** Organizes and manages UI components in a structured way, allowing proper rendering and event handling.
- **Swift Factorial Function:** Provides a simple and efficient way to calculate the factorial of any positive integer using iterative logic.

5. Swift Program to Find Sum of All Elements of a 1-D Array

```
// One-dimensional array
```

```swift
let numbers = [5, 10, 15, 20, 25]
// Variable to store sum
var sum = 0
// Iterate through each element and add to sum
for number in numbers {
    sum += number
}
// Print the sum
print("Sum of all elements: $sum)")
```

6. UI Controls Used in iOS Programming

UI controls are interface elements provided by iOS to allow user interaction with an application. They are mainly available through the **UIKit framework**.

Common Types of UI Controls

1. UILabel

- Used to display static text on the screen.
- Text cannot be edited by the user.

2. UIButton

- Used to trigger actions when tapped by the user.
- Commonly used for submit, login, and navigation actions.

3. UITextField

- Allows users to enter single-line text.
- Used for inputs like username, email, etc.

4. UITextView

- Used for multi-line text input or display.
- Supports scrolling and formatting.

5. UISwitch

- Represents an ON/OFF toggle option.
- Used for settings and preferences.

6. UISlider

- Allows users to select a value by sliding a thumb.
- Used for volume, brightness control.

7. UIImageView

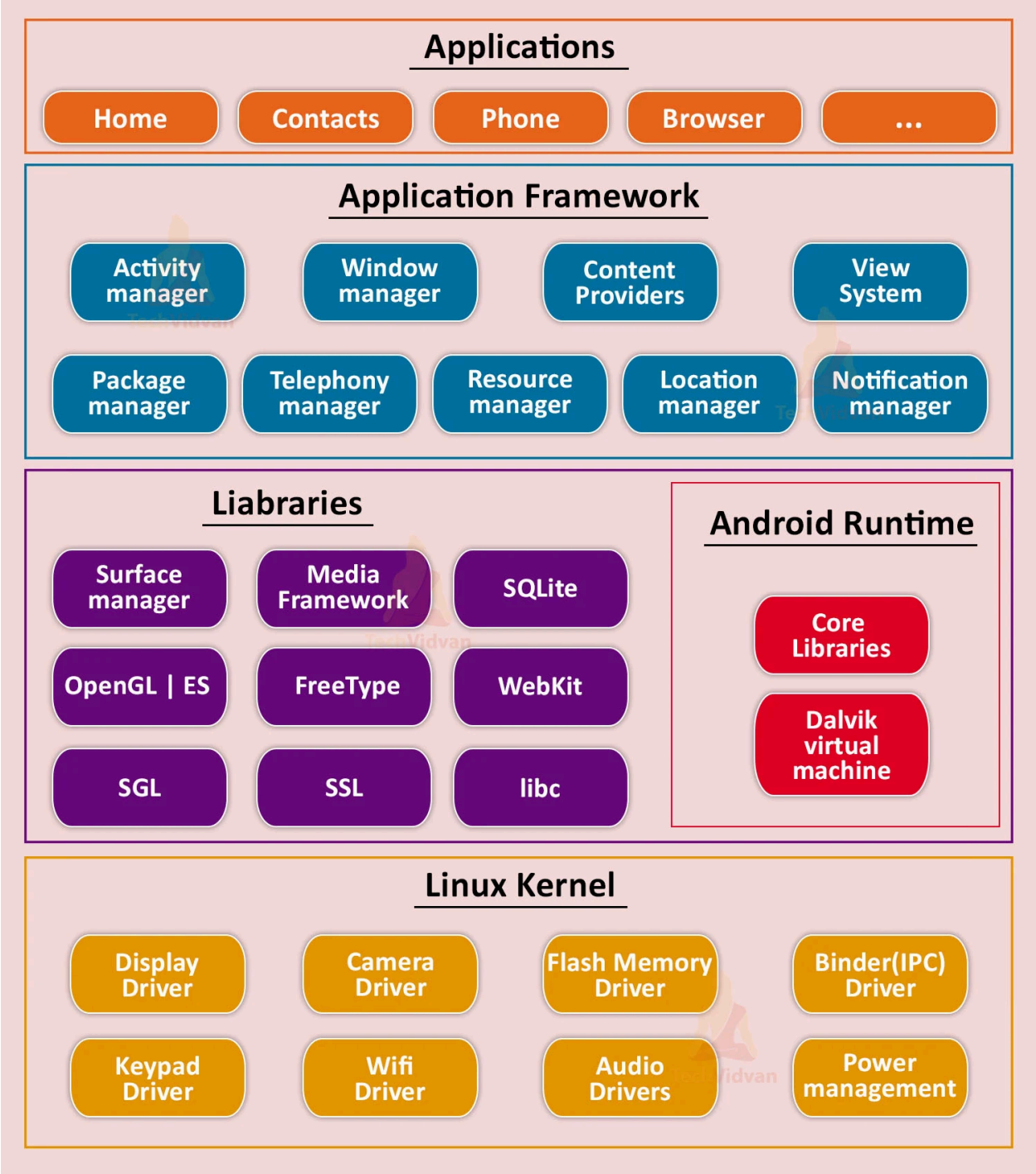
- Used to display images in an application.

Summary Table of UI Controls

UI Control	Purpose

UILabel	Display text
UIButton	Perform actions
UITextField	Single-line text input
UITextView	Multi-line text
UISwitch	ON/OFF selection
UISlider	Value selection
UIImageView	Display images

7. Android Architecture



Android architecture is a **layered software stack** that provides services and tools required to build and run Android applications. Each layer has a specific role and interacts with the layer below it.

Layers of Android Architecture

1. Applications

- Topmost layer of the architecture.
- Includes built-in apps like Phone, Contacts, Browser, and third-party apps.
- Applications are written mainly using Java or Kotlin.

---

2. Application Framework

- Provides APIs and services to developers.
- Manages application components and device features.

Key components:

- Activity Manager
- Window Manager
- Content Providers
- Resource Manager
- Notification Manager

---

3. Android Runtime (ART) and Core Libraries

- Executes Android applications.
- Core libraries provide basic functionalities like data structures, utilities, and file access.
- ART converts bytecode into native instructions for better performance.

---

4. Native Libraries

- Written in C/C++.
- Provide core system functionalities.

Examples:

- SQLite (database)
- OpenGL ES (graphics)
- Media Framework
- WebKit

---

5. Linux Kernel

- Foundation layer of Android architecture.
- Handles low-level system services.

Responsibilities:

- Process management

- Memory management
- Device drivers
- Security

8. Dialog box in android

Dialog boxes are small windows that prompt the user to make decisions or enter additional information. Common types include:

Dialog Type	Description	Example
AlertDialog	Displays messages with buttons like OK, Cancel	Confirm exit, show warnings
DatePickerDialog	Allows the user to select a date	Picking a birthdate
TimePickerDialog	Allows the user to select a time	Setting alarms
ProgressDialog	Shows progress of a task (deprecated in newer APIs)	File download, loading data
Custom Dialog	Fully customizable dialog with your own layout	Login form, input form

9. What is Surface Manager library?

The **Surface Manager** is a part of Android's **graphics architecture** that manages the display of windows and surfaces on the screen.

Key Points:

- Manages **buffers** for drawing and displaying content.

- Handles **screen composition**, ensuring multiple apps can render their content efficiently.
- Provides **smooth graphics rendering** and transitions between apps.

Functions:

- Manages surfaces for applications.
- Handles interactions between hardware and UI.
- Supports double buffering to reduce flicker.

10. How is DVM different from ART?

Feature	DVM (Dalvik Virtual Machine)	ART (Android Runtime)
Execution	Interprets bytecode at runtime	Ahead-of-Time (AOT) compilation into native code
Performance	Slower due to runtime interpretation	Faster due to precompiled native code
Memory Usage	Higher memory usage	Lower memory usage
Garbage Collection	Simple GC, may cause pauses	Improved GC, smoother performance
Introduced	Android 2.0 – 4.4	Android 5.0 and later

Summary:

- **DVM** runs bytecode directly at runtime, which is slower.
- **ART** compiles apps ahead of time into native code, improving performance and reducing battery usage.

The **Surface Manager** ensures efficient rendering of multiple surfaces and smooth UI display in Android.
ART provides faster, optimized execution compared to **DVM**, improving app performance and overall user experience.

11. Data Types Supported by SQLite

SQLite uses **dynamic typing**, but supports the following **storage classes**:

Data Type	Description	Example
INTEGER	Whole numbers, can be 1, 2, 4, 8 bytes depending on size	10, -5
REAL	Floating point numbers, 8-byte IEEE	3.14, -0.5
TEXT	Text string stored using UTF-8, UTF-16	"Hello", "Android"
BLOB	Binary data, stored exactly as input	Images, files
NULL	Represents a NULL value	NULL

Note: SQLite does not enforce strict data types like other RDBMS; a column declared as INTEGER can store a TEXT value.

12. Generating a Signed APK in Android

Steps to Generate Signed APK

1. Open the Android project in **Android Studio**
2. Click on **Build** → **Generate Signed Bundle / APK**
3. Select **APK** and click **Next**
4. Choose an existing **Keystore** or create a new one
5. Enter **Keystore password, Key alias, and Key password**
6. Select **Build Variant (release)**
7. Click **Finish**

✔ The signed APK will be generated in the `app/release/` folder

✔ Signed APK is required for publishing apps on Google Play Store

13. Android program to locate user's current location in android

MainActivity.java

```
package com.example.locationapp;
import android.Manifest;
import android.content.pm.PackageManager;
import android.location.Location;
```

```java
import android.location.LocationListener;
import android.location.LocationManager;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
public class MainActivity extends AppCompatActivity implements
LocationListener {
    LocationManager locationManager;
    TextView tvLocation;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        tvLocation = findViewById(R.id.tvLocation);
        locationManager = (LocationManager)
getSystemService(LOCATION_SERVICE);
        if (ActivityCompat.checkSelfPermission(this,
                Manifest.permission.ACCESS_FINE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this,
                    new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, 1);
            return;
        }
        locationManager.requestLocationUpdates(
                LocationManager.GPS_PROVIDER, 0, 0, this);
    }
    @Override
    public void onLocationChanged(Location location) {
        double latitude = location.getLatitude();
        double longitude = location.getLongitude();
        tvLocation.setText("Latitude: " + latitude + "\nLongitude: " + longitude);
    }
}
```

AndroidManifest.xml

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
   package="com.example.locationapp">
   <uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION"/>
   <uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION"/>
   <application
      android:allowBackup="true"
      android:label="Location App"
      android:theme="@style/Theme.AppCompat.Light.DarkActionBar">
      <activity android:name=".MainActivity">
         <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
         </intent-filter>
      </activity>
   </application>
</manifest>
```

14. Types of Mobile Application Development Approaches

Approach	Description	Advantages	Disadvantages	Example

Native App Development	Apps are developed **specifically for a platform** using platform-specific languages and SDKs.	High performance, full access to device features, smooth UI/UX	Requires separate code for each platform, higher cost	Android: Java/Kotlin, iOS: Swift/Objective-C
Web App Development	Apps are developed using **HTML, CSS, and JavaScript**, and run in mobile browsers.	Cross-platform, easier to maintain, no installation required	Limited access to device features, dependent on internet	Mobile websites, Progressive Web Apps (PWA)
Hybrid App Development	Combines **web technologies** wrapped in a native container. Works across multiple platforms.	Single codebase, faster development, access to some device features	Performance may be lower than native, limited hardware access	Ionic, Cordova, React Native
Cross-Platform Development	Uses frameworks to **write one codebase** that runs on multiple platforms natively.	Faster development, maintainable code, near-native performance	Some platform-specific adjustments required	Flutter, Xamarin, React Native

---

Detailed Explanation

1. Native App Development

- Written using **platform-specific languages** and SDKs.
- **Pros:** Best performance, full access to camera, GPS, sensors, push notifications.
- **Cons:** Need to maintain separate code for Android and iOS, higher development cost.

2. Web App Development

- Runs inside a **mobile browser**, no installation needed.
- **Pros:** Works on any device with a browser, easy updates.
- **Cons:** Cannot access all device hardware, requires internet connection.

3. Hybrid App Development

- Uses **web technologies** (HTML, CSS, JS) inside a **native wrapper**.
- **Pros:** Single codebase, moderate access to device features, faster development.
- **Cons:** Performance may lag for graphics-heavy apps, limited access to some native APIs.

4. Cross-Platform Development

- Frameworks like **Flutter, React Native, Xamarin** allow one codebase for multiple platforms.
- **Pros:** Near-native performance, easier maintenance, cost-effective.
- **Cons:** May require native code for complex features, larger app size.

Comparison Table

Feature	Native	Web	Hybrid	Cross-Platform
Performance	Excellent	Moderate	Moderate	Near-native
Device Access	Full	Limited	Partial	Good
Development Cost	High	Low	Medium	Medium
Maintenance	Separate for each platform	Single	Single	Single
User Experience	Best	Limited	Moderate	Good

15. Procedure to Generate Google Map API Key

1. Sign in to Google Cloud Platform (GCP):

- Go to https://console.cloud.google.com/.

2. Create a New Project:

- Click on **Select a project** → **New Project**.
- Enter project name (e.g., `MapDemo`) and click **Create**.

3. Enable Maps SDK for Android:

- Navigate to **APIs & Services** → **Library**.
- Search **Maps SDK for Android** and click **Enable**.

4. Generate API Key:

- Go to **APIs & Services** → **Credentials** → **Create Credentials** → **API Key**.
- Copy the generated API Key.

5. Restrict API Key (Recommended):

- Click on your API key → **Application Restrictions** → **Android apps**.
- Add your **package name** and **SHA-1 fingerprint**.
- Save changes.

6. Add API Key to Android Project:

- Open `res/values/google_maps_api.xml` and replace `<YOUR_API_KEY>` with the generated API key.